Short Delivery Routes on Java

Hill Climbing vs. Simulated Annealing vs. Genetic Algorithm for the Travelling Salesman Problem

Artificial Intelligence Concepts (EK234201) - Final Report

Authors

Evann Bloutin
Mikal Zheng
Cecilia Brugnot
Coralie Pittet
Skander Mokeddem

Institut Teknologi Sepuluh Nopember (ITS), Surabaya

October 2026

Source code: https://github.com/EvannBloutin/AI_Concepts_Midterm

Chapter 1 Background, Problem Statement & Benefits

1.1 Background

Indonesia is one of the fastest-growing e-commerce markets in South-East Asia, and the island of Java concentrates more than half of the country's population and most of its parcel volume. For a courier, a large share of the delivery cost is the distance driven: fuel, driver hours, vehicle wear and CO_2 emissions all grow with the length of the route. Choosing *the order* in which a vehicle visits its stops is therefore a daily, repeated decision with a direct cost.

Stripped to its core, this decision is the **Travelling Salesman Problem (TSP)**: given n cities and the distance between every pair, find the shortest closed tour that visits each city exactly once and returns to the start. The TSP is NP-hard; a symmetric instance has $(n\text{-}1)!/2$ distinct tours, i.e. about 2×10^7 for 12 cities and 4×10^{30} for 30 cities. Exhaustive or classical tree search (Uniform-Cost Search, A*) is exact but explodes quickly, which is exactly the situation in which the course introduces **local search**: keep a single complete solution and improve it by small changes, accepting that the result may not be provably optimal.

The course presents three such algorithms: **Hill Climbing** (with random restarts), **Simulated Annealing** and the **Genetic Algorithm**. They share the same ingredients (a state, a neighbourhood, an objective) but differ in how they escape local optima: by restarting, by sometimes accepting worse moves, or by recombining a population of solutions. This project compares them on the same problem, under the same budget.

1.2 Problem Statement

We study a single-vehicle delivery tour through real cities of Java (Jakarta to Banyuwangi, great-circle distances in km), complemented by random Euclidean instances of 50 and 100 points to test scalability. The project answers four questions:

RQ1 - Up to which size can exact search (brute force, UCS, A*, dynamic programming) solve the problem, and how much does an admissible heuristic help?

RQ2 - Given the *same number of evaluated candidate solutions*, which of Hill Climbing with random restarts (HC-RR), Simulated Annealing (SA) and the Genetic Algorithm (GA) finds the shortest tours, and how reliably?

RQ3 - How do the answers change with instance size, evaluation budget and the algorithms' parameters?

RQ4 - Does hybridising the GA with hill climbing (a memetic algorithm, MA) help under a fixed budget?

1.3 Benefits

- **Practical**: on the 30-city Java route, local search finds a tour of 2060 km, versus 2289 km for the greedy nearest-neighbour rule a dispatcher might use by hand - about 10 % less driving, in well under a second of computation.
- **Methodological**: a fair, reproducible comparison (identical evaluation budget, 30 seeded runs, statistical tests, exact optima where they can be computed) gives evidence rather than anecdote on which algorithm to choose.
- **Educational**: the experiments make the lecture concepts measurable - the combinatorial explosion of uninformed search, the effect of an admissible heuristic, the probability that a single hill climb succeeds and the expected number of restarts ($1/p$), the role of the cooling schedule, and the cost of a population.
- **Reusable**: the code is a small, dependency-free Python package with unit tests, so it can be extended to other cities or to richer routing problems.

Chapter 2 Method

2.1 Problem formulation

Let $d(i,j)$ be the distance between cities i and j. A **state** is a permutation π of the n cities, and the **objective** to minimise is the closed-tour length $L(\pi) = \Sigma_k\, d(\pi_k, \pi_{k+1 \bmod n})$. Every permutation is a valid tour, so all algorithms work on complete solutions.

The **successor function** is the classical **2-opt move** (Croes, 1958): remove two edges (a,b) and (c,e) of the tour and reconnect it as (a,c) and (b,e), which reverses the segment between them. It is the TSP analogue of moving one queen in the 8-queens example of the lecture. A tour has $n(n\text{-}3)/2$ such neighbours, and the change in length is computed in O(1) without re-evaluating the tour:

```
Δ = d(a,c) + d(b,e) - d(a,b) - d(c,e)
```

2.2 Instances

Table 1 - Problem instances. The two Java instances use approximate city-centre coordinates; the random instances are generated with a fixed seed.

Instance	n	Type	Distance	Reference value
Java-12	12	12 major cities of Java	great-circle km	1907.47 - exact optimum (Held-Karp)
Java-30	30	30 cities of Java	great-circle km	2060.35 - best of 5 long SA runs (10× budget)
U-50	50	uniform random, 1000×1000 square	Euclidean	6078.21 - best of 5 long SA runs (10× budget)
U-100	100	uniform random, 1000×1000 square	Euclidean	7727.55 - best of 5 long SA runs (10× budget)

2.3 Exact search (ground truth)

On small instances the true optimum is computed in four ways. **Brute force** enumerates all $(n\text{-}1)!/2$ tours. **Uniform-Cost Search (UCS)** and **A*** search a graph whose states are (set of visited cities, current city), starting from city 0; the goal is reached when all cities are visited and the tour is closed. A* uses the **minimum-spanning-tree heuristic**: the cost of an MST over the unvisited cities, the current city and city 0. Any completion of the tour is a spanning path over these cities, which costs at least the MST, so the heuristic is *admissible* and A* returns the optimum. **Held-Karp** dynamic programming solves the same problem in $O(n^2 2^n)$ time. For larger instances we report the classical **1-tree lower bound** (MST without city 0 plus its two cheapest edges).

2.4 Hill Climbing with random restarts (HC-RR)

We use **first-choice hill climbing** (Russell & Norvig, sec. 4.1.1): the neighbours are visited in a random cyclic order and the first improving 2-opt move is applied. Steepest-ascent would spend $n(n-3)/2$ evaluations per step. A **local optimum** is detected exactly, when a full cycle through all neighbours yields no improvement. While budget remains, the search **restarts** from a new random tour, and the best local optimum is returned.

2.5 Simulated Annealing (SA)

SA picks a random 2-opt neighbour; an improving move is always accepted and a worsening move is accepted with probability $\exp(-\Delta/T)$ (Kirkpatrick et al., 1983). The temperature follows a **geometric schedule** $T \leftarrow \alpha T$. Both ends of the schedule are derived from the instance and the budget instead of being hand-tuned: T_0 is chosen so that an average uphill move from a random tour is accepted with probability $p_0 = 0.3$, i.e. $T_0 = -\text{mean}(\Delta^+)/\ln p_0$, and α is set so that the temperature reaches $10^{-3}T_0$ exactly when the budget is spent. The best tour ever visited is returned.

2.6 Genetic Algorithm (GA)

The GA follows the lecture's loop *selection* → *crossover* → *mutation* with a permutation encoding. Fitness is the tour length (lower is better). Parents are chosen by **tournament selection** ($k = 3$): roulette-wheel selection on $1/L$ is almost uniform when all tours have similar lengths. One-point crossover would create invalid tours (duplicated cities), so we use **order crossover (OX)** (Davis, 1985): a random segment of parent 1 is kept and the remaining cities are filled in the order in which they appear in parent 2. **Mutation** reverses a random segment (a random 2-opt move). Replacement is generational with **elitism** (the 2 best survive). Parameters: population 100, crossover rate 0.9, mutation rate 0.5.

2.7 Memetic Algorithm (MA, extension)

The MA (Moscato, 1989) is the GA in which each child, with probability 0.5, is improved by a short first-choice 2-opt hill climb of at most n neighbour evaluations; the improved child replaces the original (Lamarckian evolution). Every evaluation of this local search is charged to the same budget as all other algorithms.

2.8 Baselines

Random search (RS) returns the best of B random tours and shows what the same budget achieves without any guidance. **Nearest neighbour (NN)** is the greedy constructive rule "always drive to the closest unvisited city", run from every start city; it represents a reasonable manual plan.

2.9 Experimental protocol

Fair budget. Every algorithm receives the same budget of $B = 10\,000 \times n$ *candidate evaluations*: one 2-opt neighbour tested (HC, SA), one child evaluated (GA), or one random tour generated (RS). This counts search effort independently of the machine. Wall-clock time is reported as well, because a GA child costs an O(n) tour evaluation while a 2-opt delta costs O(1).

Runs and seeds. Each algorithm is run 30 times per instance (10 for RS) with seeds 1000-1029, shared by all algorithms. Parameters were fixed beforehand on an independent pilot instance (random, $n = 50$, seed 999), not on the test instances.

Metrics. Best, median, mean and worst tour length; the **gap** to the reference value, $100\times(L/L_{\text{ref}} - 1)$ %; the number of runs that reach the reference; and the run time. Differences are tested with the one-sided Mann-Whitney U test, since the length distributions are not normal.

Experiments. E1 exact search for $n = 6..12$; E2 main comparison on the four instances; E3 effect of the budget (factor 100 to 100 000 per city on U-50); E4 1000 single hill climbs to measure the success probability p of one climb; E5 parameter sensitivity of SA and GA on Java-30. All code is Python 3 standard library; 16 unit tests check tour validity, the 2-opt delta formula and that A*, UCS, brute force and Held-Karp agree.

Chapter 3 Experiment & Discussion

3.0 Implementation

GitHub: https://github.com/EvannBloutin/AI_Concepts_Midterm

The solvers are written in plain Python 3 (standard library only) and organised as a small package in `src/`. Every solver has the same interface `solver(dist, budget, seed, params)` and returns the best tour, its length, the number of evaluations used and the best-so-far history. The raw results of every run are stored as JSON in `results/`, from which all tables and figures of this chapter are drawn.

Table 2 - Structure of the implementation.

Module	Content
src/tsp/core.py	distances, tour length, 2-opt move and O(1) delta, 2-opt local search
src/tsp/data.py	instances Java-12, Java-30 (real cities), U-50, U-100 (random)
src/tsp/hill_climbing.py	first-choice hill climbing with random restarts
src/tsp/annealing.py	simulated annealing, calibrated geometric cooling
src/tsp/genetic.py	genetic algorithm (tournament, OX, inversion, elitism) + memetic
src/tsp/baselines.py	random search, nearest neighbour
src/tsp/exact.py, astar.py	brute force, Held-Karp, 1-tree bound; UCS / A* (MST)
src/experiments/	one module per experiment group (E1-E6), shared seeds and parameters
tests/	16 unit tests (core, exact solvers, heuristics)

To reproduce: `pip install -r requirements.txt`, then `python -m unittest discover -s tests` and `python src/run_experiments.py` (about 40-50 minutes on one CPU core). All runs are seeded, so the numbers below are reproduced exactly; only wall-clock times depend on the machine.

3.1 Exact search and the combinatorial explosion (RQ1)

Table 3 - Exact search on the first n cities of Java-12. All four methods return the same optimum (checked by the unit tests).

n	tours (n-1)!/2	Brute force (s)	UCS expanded	A* expanded	A* / UCS	Held-Karp (s)	Optimum (km)
6	60	0.000	81	42	52 %	0.000	994.1
7	360	0.001	193	66	34 %	0.001	1078.2
8	2,520	0.006	449	102	23 %	0.002	1122.9
9	20,160	0.030	1,025	241	24 %	0.002	1261.6
10	181,440	0.193	2,305	585	25 %	0.007	1520.3
11	1,814,400	-	5,121	729	14 %	0.011	1586.4
12	19,958,400	-	11,265	2,154	19 %	0.025	1907.5

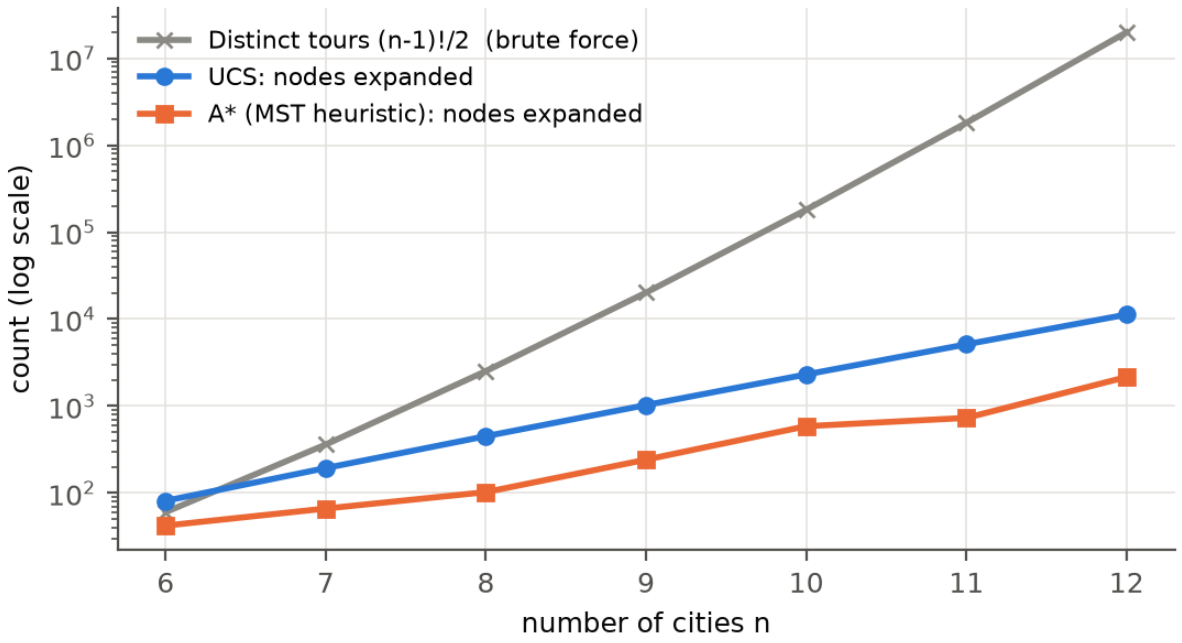


Figure 1 - Search effort of exact methods versus problem size.

The number of tours grows factorially: brute force already needs 0.19 s for 10 cities, and extrapolating its rate gives about 13 hours for 15 cities, which is why it is stopped at $n = 10$. UCS works on (set, city) states, of which there are only $n \cdot 2^n$, so it remains fast up to 12 cities; still, its expansions double with every added city. The admissible MST heuristic cuts the expansions of A* to 19 % of UCS at $n = 12$ (2,154 vs. 11,265) and the saving grows with n. In wall-clock time, however, A* is not faster here, because each heuristic evaluation computes an MST. Both remain exponential: for 30 cities, $30 \cdot 2^{30} \approx 3 \times 10^{10}$ states are out of reach, which motivates local search. The Java-12 optimum (1907.5 km) is the reference for E2.

3.2 Main comparison under an equal budget (RQ2)

Table 4 - Tour length over 30 runs (10 for RS; NN is deterministic). Gap is relative to the reference of Table 1; Hits = runs reaching the reference value. Units: km for Java, plane units for U-50/U-100.

Instance	Algorithm	Best	Median	Worst	Median gap	Hits	Time (s)
Java-12	NN	2014.1	2014.1	2014.1	5.59 %	-	0.00
	RS	1939.9	1988.4	2045.9	4.24 %	0/10	0.58
	HC-RR	1907.5	1907.5	1907.5	0.00 %	30/30	0.08
	SA	1907.5	1907.5	1907.5	0.00 %	30/30	0.11
	GA	1907.5	1907.5	1907.5	0.00 %	30/30	0.84
	MA	1907.5	1907.5	1907.5	0.00 %	30/30	0.36
Java-30	NN	2289.0	2289.0	2289.0	11.10 %	-	0.00
	RS	5150.2	5365.1	5768.5	160.40 %	0/10	3.24
	HC-RR	2060.3	2060.3	2060.3	0.00 %	30/30	0.17
	SA	2060.3	2060.3	2060.3	0.00 %	30/30	0.30
	GA	2060.3	2060.3	2060.3	0.00 %	30/30	2.72
	MA	2060.3	2060.3	2079.1	0.00 %	28/30	1.76
U-50	NN	6791.2	6791.2	6791.2	11.73 %	-	0.01
	RS	18600.5	19234.1	19867.0	216.44 %	0/10	9.12
	HC-RR	6087.0	6148.3	6201.4	1.15 %	0/30	0.29
	SA	6078.2	6111.8	6287.6	0.55 %	6/30	0.54
	GA	6078.2	6224.0	6443.4	2.40 %	6/30	5.99
	MA	6087.0	6297.1	6560.7	3.60 %	0/30	4.84
U-100	NN	9105.8	9105.8	9105.8	17.84 %	-	0.10
	RS	40005.2	40580.1	41515.5	425.14 %	0/10	36.37
	HC-RR	8032.1	8204.3	8329.9	6.17 %	0/30	0.60
	SA	7758.9	7891.4	8138.2	2.12 %	0/30	1.13
	GA	7925.8	8191.4	8462.9	6.00 %	0/30	18.59
	MA	7964.0	8696.1	9444.0	12.53 %	0/30	20.63

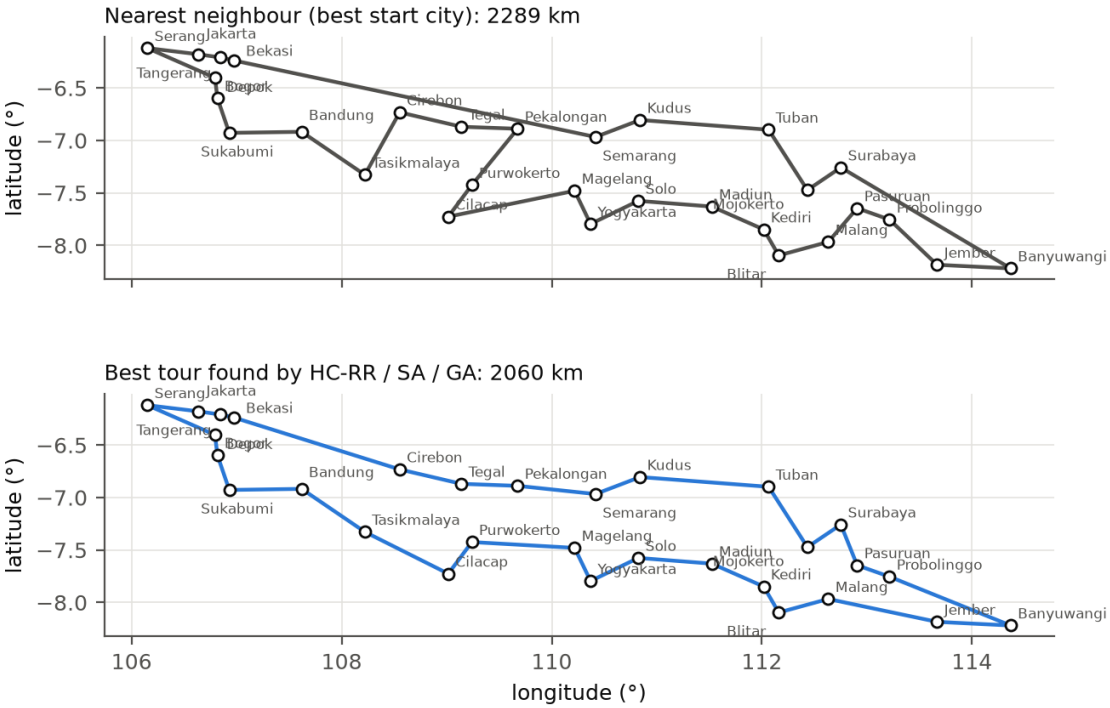


Figure 2 - Java-30: the greedy nearest-neighbour tour (top) has a long jump back across the island and a zig-zag in Central Java; the tour found by every local-search run (bottom) follows the north coast eastwards and returns along the south.

Small instances. On Java-12 and Java-30 all four local-search algorithms reach the reference value in every run (the MA misses it in 2 of 30 runs on Java-30). On Java-12 this value is the proven optimum. On Java-30 it is the best of 5 long SA runs; all algorithms agree on it independently, which makes it very likely optimal as well. Random search, on the contrary, is 160 % above it on Java-30: guidance by neighbourhood, not the number of evaluations, makes the difference. Nearest neighbour is 5-18 % above the reference, and its gap grows with *n*.

Larger instances. The algorithms separate on U-50 and U-100 (Figure 3). **Simulated annealing is the best algorithm on both**: median gap 0.55 % on U-50 and 2.12 % on U-100. On U-100 it beats HC-RR, GA and MA with $p < 10^{-7}$ (one-sided Mann-Whitney, p = 5e-11, 1e-08, 4e-11); even its worst run on U-100 (8138) is better than the median of HC-RR (8204). On U-50 SA beats GA (p = 0.001) and MA (p = 6e-08), but its advantage over HC-RR is not significant (p = 0.17). HC-RR is very consistent (smallest spread) but never reaches the reference on the random instances. The GA reaches it in 6 of 30 runs on U-50, as often as SA, but has the largest spread, and on U-100 it only ties with HC-RR (p = 0.74, two-sided) while taking 30 times longer.

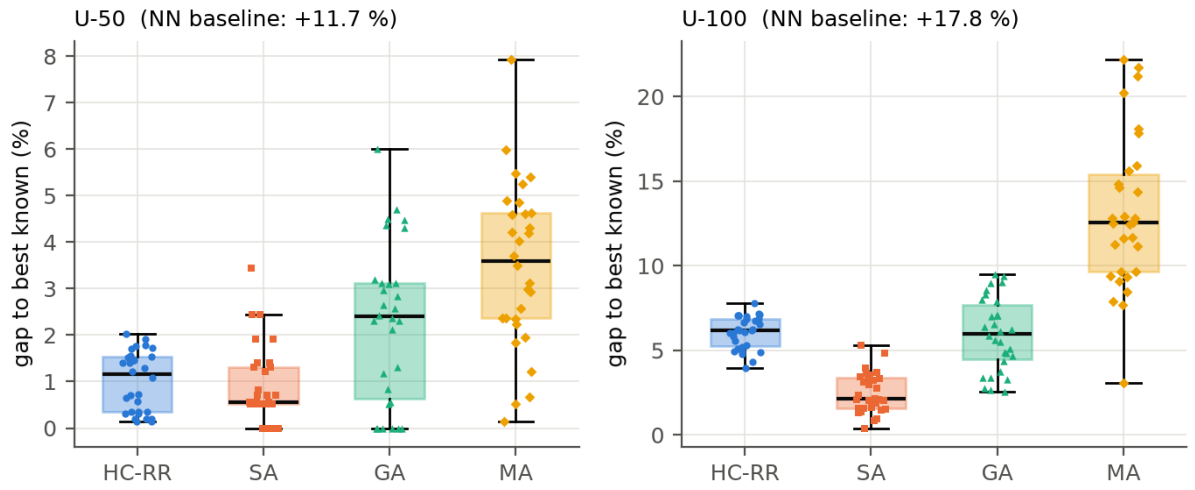


Figure 3 - Distribution of the gap over 30 runs on the random instances (box = quartiles, line = median, dots = individual runs).

Run time. At equal evaluation count, HC-RR and SA need 0.3-1.1 s, while GA and MA need 5-21 s on U-50/U-100, because each GA evaluation is a full O(n) tour length plus an O(n) crossover, whereas a 2-opt move is evaluated in O(1). Under equal *time* the gap between SA and the GA would therefore be even larger.

3.3 Convergence behaviour

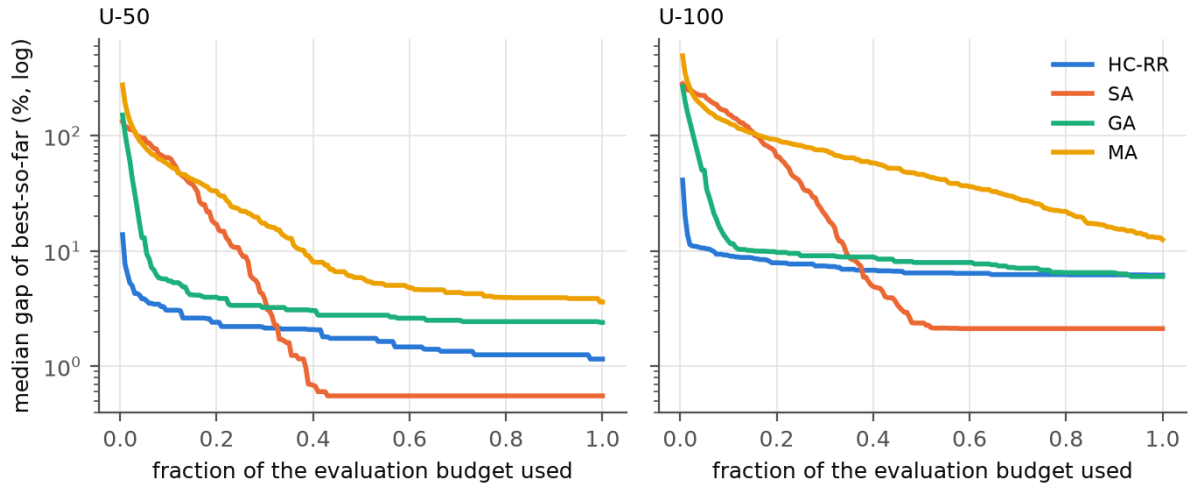


Figure 4 - Median best-so-far gap (log scale) during the run, 30 runs, as a function of the fraction of the budget used.

Figure 4 shows the different search strategies. HC-RR improves fastest at first, since its first climb ends at a local optimum within a few thousand evaluations, and then progresses only by luck through restarts: the curve flattens after about 10 % of the budget. SA is *worse* than HC-RR for most of the run, because at high temperature it deliberately accepts bad moves (exploration), and only overtakes it after about a third of the budget (30 % on U-50, 35 % on U-100), when the temperature drops and it turns into a greedy descent from a good region (exploitation). The GA improves quickly during the first 10 % of the budget, while the random initial population is being sorted out, and then stagnates: once the population has converged, a child of two good parents is rarely better than both. This trade-off between exploration and exploitation is the central idea of the lecture, and it is clearly visible in the data.

3.4 Effect of the budget (RQ3)

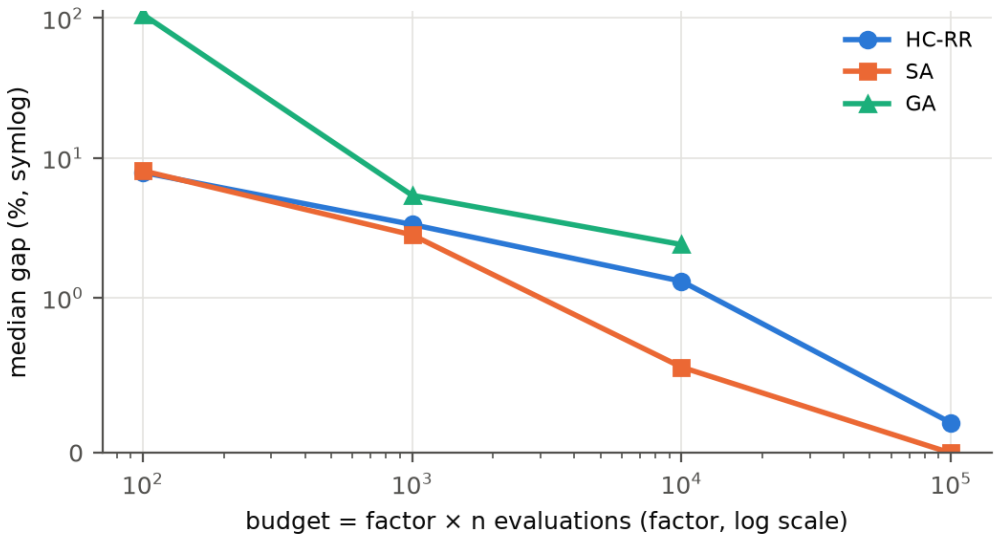


Figure 5 - Median gap on U-50 (20 runs per point) as a function of the budget. GA is not run at factor 100 000 (too slow in pure Python).

With a tiny budget (100×*n*) HC-RR and SA perform the same (about 8 % gap) because SA has no time to anneal, and the GA has barely started (106 % gap), since a population needs many generations. As the budget grows, all methods improve, but SA improves most: at 100 000×*n* its median *equals* the reference value (6078.2), reached in 14 of 20 runs, while HC-RR reaches it in 2 of 20 and its median is still 0.19 % above. For SA, slower cooling directly gives better tours, which matches the theory (with a slow enough schedule SA converges to a global optimum). For HC-RR, more budget only means more independent samples of local optima, and the returns diminish quickly.

3.5 How often does one hill climb succeed?

Table 5 - Probability p that a single hill climb reaches the reference value (or comes within a tolerance of it). Measured restarts: mean over 100 repetitions of 'restart until success'.

Instance	Evals / climb	Threshold	p (1000 climbs)	Expected restarts 1/p	Measured restarts
Java-30	1590	reference	0.414	2.4	2.6
		≤ +1 %	0.618	1.6	1.7
		≤ +2 %	0.662	1.5	1.5
		≤ +5 %	1.000	1.0	1.0
U-50	4982	reference	0.000	∞	-
		≤ +1 %	0.003	333.3	-
		≤ +2 %	0.015	66.7	38.9
		≤ +5 %	0.260	3.8	3.9

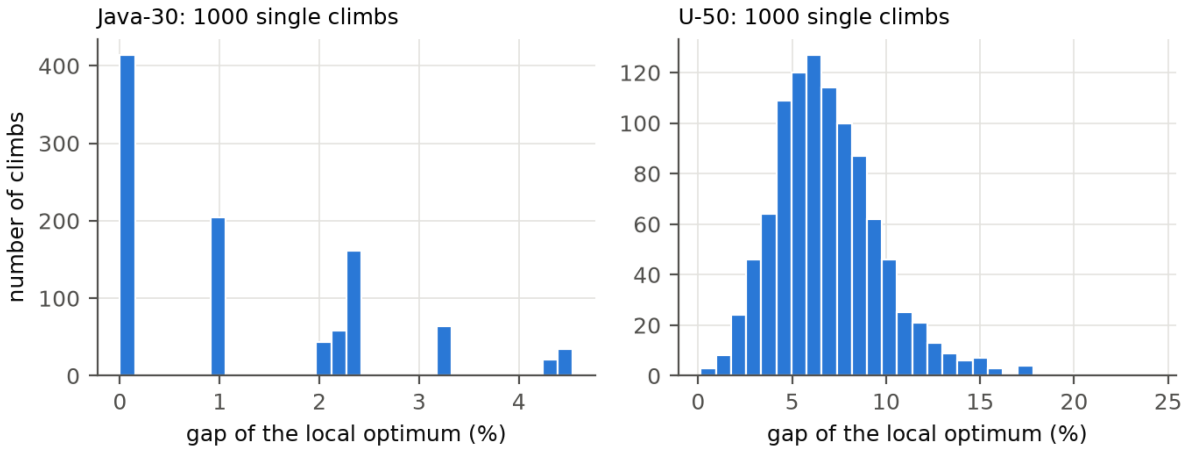


Figure 6 - Quality of 1000 single 2-opt local optima.

The lecture states that hill climbing with random restarts needs on average $1/p$ restarts, where p is the probability that one climb succeeds. Table 5 confirms this: on Java-30, $p = 0.414$, predicting 2.4 restarts, and 2.6 were measured. This explains why HC-RR is perfect on the Java instances: the 300 000 evaluation budget allows about 190 climbs. On U-50, however, none of the 1000 climbs reaches the reference and only 0.3 % come within 1 %. The landscape has many more local optima, and restarting no longer helps, because each restart starts from scratch and forgets the good regions found before. SA and the GA keep this information (the current tour, the population), which is why they win as n grows.

3.6 Parameter sensitivity

Table 6 - SA on Java-30: median length in km (runs reaching 2060 km out of 10) for each initial acceptance probability p_0 and final temperature.

p_0 \ T_{end}/T_0	10^{-1}	10^{-2}	10^{-3}	10^{-4}
0.8	2915 (0/10)	2060 (9/10)	2060 (10/10)	2060 (10/10)
0.3	2074 (3/10)	2060 (10/10)	2060 (10/10)	2060 (10/10)
0.1	2060 (10/10)	2060 (10/10)	2060 (10/10)	2060 (10/10)

Table 7 - GA on Java-30: median length in km (runs reaching 2060 km out of 10).

population \ mutation	0.05	0.2	0.5	0.8
20	2070 (5/10)	2060 (8/10)	2060 (10/10)	2060 (10/10)
50	2060 (7/10)	2060 (6/10)	2060 (10/10)	2060 (9/10)
100	2060 (7/10)	2060 (7/10)	2060 (10/10)	2060 (9/10)
200	2060 (6/10)	2060 (6/10)	2060 (10/10)	2060 (6/10)

Both algorithms are robust on Java-30, and almost every setting finds the reference tour. SA only fails when it starts hot ($p_0 = 0.8$) *and* stops hot ($T_{end} = 0.1T_0$): it never reaches the low-temperature phase in which it becomes a descent, and it ends near a random walk. Because T_0 and α are derived from the instance and the budget, the schedule does not need to be re-tuned per instance. The GA degrades when a small population is combined with low mutation (20 / 0.05): diversity is lost and the population converges prematurely, which is the classic GA failure mode.

3.7 Discussion

Why SA wins. SA combines the cheap O(1) 2-opt evaluation of hill climbing with a controlled way of escaping local optima. Unlike restarts, an uphill move keeps most of the current tour, so SA explores *around* good solutions instead of starting from random ones. Its main cost is that it needs a sufficient budget: with very few evaluations it is no better than hill climbing (Figure 5).

Why the GA is weaker here. The TSP objective depends on *adjacencies*, and OX mostly preserves the *relative order* of cities, so offspring often lose good edges of both parents. The GA also spends its budget on a whole population and is 10-30 times slower in wall-clock time. Its strength is diversity: it reached the reference on U-50 as often as SA, and it never collapses to one local optimum, which is why its spread is the largest.

Why the memetic variant did not help (RQ4). The MA is worse than the plain GA on U-50 and U-100, against what the literature often reports. Our local search is capped at n evaluations per child, while a full 2-opt descent needs on the order of n^2 evaluations; each child therefore gets a partial improvement that costs budget (fewer generations) and, being Lamarckian, pushes the population towards similar tours (less diversity). A full descent on fewer children, or an O(1)-delta implementation of the GA's mutation, would be the next thing to try; we report the negative result as it is.

Limitations. (i) The reference values for Java-30, U-50 and U-100 are not proven optima; the 1-tree lower bounds (1576, 5328 and 6977) are too loose to close the gap, since the bound is 30 % below the true optimum even on Java-12. A solver such as Concorde would give certified optima. (ii) Distances are straight-line great-circle distances, not road distances; the road network of Java would change the values but not the method. (iii) Parameters were tuned once on a pilot instance; a full tuning per algorithm could change some margins, but Tables 6 and 7 show that the results are not sensitive to them. (iv) The implementation is pure Python, so absolute times are slower than a compiled implementation; only relative times should be compared.

Chapter 4 Conclusion

We formulated the planning of a delivery tour through the cities of Java as a Travelling Salesman Problem and compared the three local-search algorithms of the course, plus a memetic extension, against exact search and simple baselines, using an equal evaluation budget and 30 seeded runs per configuration.

- **RQ1 - Exact search** is only practical for very small routes. Brute force stops at about 10 cities; UCS and A* on (visited set, city) states reach 12 cities easily, and the admissible MST heuristic reduces the number of expanded nodes by a factor of 5.2, but the growth stays exponential.
- **RQ2 - Simulated annealing is the best overall choice.** All local-search methods solve the 12- and 30-city Java routes in every run, cutting the nearest-neighbour route by 10 % on Java-30. On 100 cities SA is significantly better than every other algorithm (median gap 2.1 % vs. 6.0-12.5 %) and is also one of the fastest.
- **RQ3 - Size and budget** decide between the methods. Hill climbing with restarts is perfect when one climb succeeds often ($p = 0.41$ on Java-30, i.e. about 2.4 restarts), but useless once p becomes almost zero (U-50). SA profits the most from a larger budget, while the GA needs a large budget and diversity (population and mutation) to work at all.
- **RQ4 - The memetic variant**, as implemented with a short local search, did not improve on the GA under a fixed budget.

For a courier planning a route of a few dozen stops, the practical recommendation is simple: start from the nearest-neighbour tour or a random tour, and run simulated annealing with 2-opt moves and a calibrated geometric cooling schedule. It takes less than a second and gives routes about 10 % shorter than the greedy rule.

Future work

Natural extensions are (1) real road distances and travel times from an open routing engine; (2) stronger neighbourhoods (Or-opt, 3-opt, Lin-Kernighan) and edge-preserving crossovers such as EAX for the GA; (3) certified optima with Concorde to measure the true gaps; and (4) the realistic delivery variants: several vehicles with capacities (VRP) and customer time windows (VRPTW), to which the same local-search framework applies directly.

References

Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.

Davis, L. (1985). Applying adaptive algorithms to epistatic domains. In *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI)*, 162-164.

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196-210.

Held, M., & Karp, R. M. (1970). The traveling-salesman problem and minimum spanning trees. *Operations Research*, 18(6), 1138-1162.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. In E. Aarts & J. K. Lenstra (Eds.), *Local Search in Combinatorial Optimization* (pp. 215-310). Wiley.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.

Lin, S., & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2), 498-516.

Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1), 50-60.

Moscato, P. (1989). *On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms* (C3P Report 826). California Institute of Technology.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Ch. 3 Search, Ch. 4 Local Search.)

Project source code and raw results: https://github.com/EvannBloutin/AI_Concepts_Midterm